

UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

GRUPO DE INVESTIGACION (MULTIMEDIA)

DANIEL ALEJANDRO PRESIGA

UNIVERSIDAD DISTRITAL FRANCISCO JOSE DE CALDAS

FACULTAD DE INGENIERIA

BOGOTA, COLOMBIA

2026

TABLA DE CONTENIDO

IDEA 3
PROBLEMA IDENTIFICADO 3
OBJETIVO GENERAL 4
OBJETIVOS ESPECIFICOS 4
FIGURA 1 (DIAGRAMA DE FLUJO) 5
FIGURA 2 (DIAGRAMA DE CASO DE USO EN EL BUSCADOR) 6
FIGURA 3 (DIAGRAMA DE CASO DE USO CODIGO PROPIO) 7
FIGURA 4 (DIAGRAMA DEL SISTEMA) 8
PLANTEAMIENTO Y USO DE PROMPTS 9
PARTES MAS RELEVANTES DEL CODIGO 13
EVIDENCIAS DE LO OBTENIDO CON EL CODIGO 18
TABLA COMPARATIVA 24
CONCLUSIONES 25
BIBLIOGRAFIA 27

IDEA

Investigar la capacidad de las técnicas de la inteligencia artificial “GEMINI IA” para generar un conjunto de términos semánticamente relacionados a partir de un concepto central. El interés no se limita exclusivamente a la generación automática de las palabras, sino también al análisis de su pertinencia, coherencia y relación al contexto planteado.

Como caso de aplicación, los términos obtenidos serán utilizados para que la misma IA genera automáticamente una herramienta educativa didáctica (sopa de letras), dicha sopa de letras será comparada con la generada por un recurso (código) de creación propia, la generación de ambas herramientas se hace con fines recreativos y esto nos permitirá evaluar y comparar la utilidad de las estrategias empleadas.

PROBLEMA IDENTIFICADO

Aunque las herramientas basadas en inteligencia artificial permiten generar palabras relacionadas con un concepto determinado, no siempre existe una garantía de que los términos generados por estas sean pertinentes, coherentes entre si o adecuados para el contexto en el que serán utilizados.

En el caso de recursos didácticos como las sopas de letras, la selección incorrecta de términos puede afectar la calidad y utilidad de la herramienta. Por esta razón surge la necesidad de investigar y verificar como la IA, en este caso “GEMINI IA” pueden generar y seleccionar conjuntos de palabras semánticamente relacionadas, así como establecer criterios que permitan evaluar la calidad y pertinencia de los resultados obtenidos en contraste con lo que obtenido con el código propio.

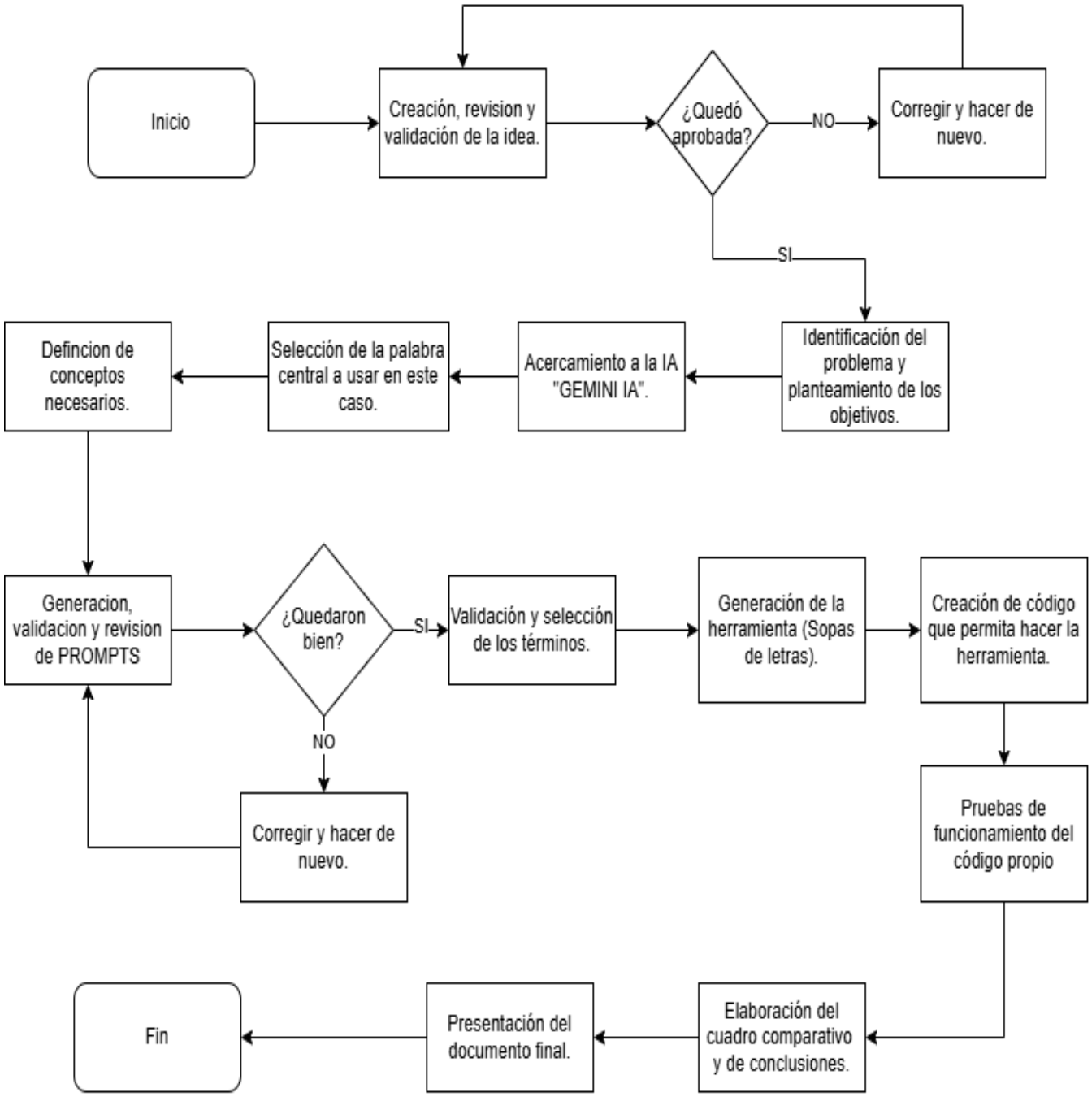
OBJETIVO GENERAL

Investigar, evaluar y comparar la capacidad de la IA “GEMINI IA” para generar y seleccionar términos semánticamente relacionados a partir de un concepto central, analizando su pertinencia, coherencia y adecuación al contexto, para su posterior aplicación en la generación automática de sopas de letras.

OBJETIVOS ESPECIFICOS

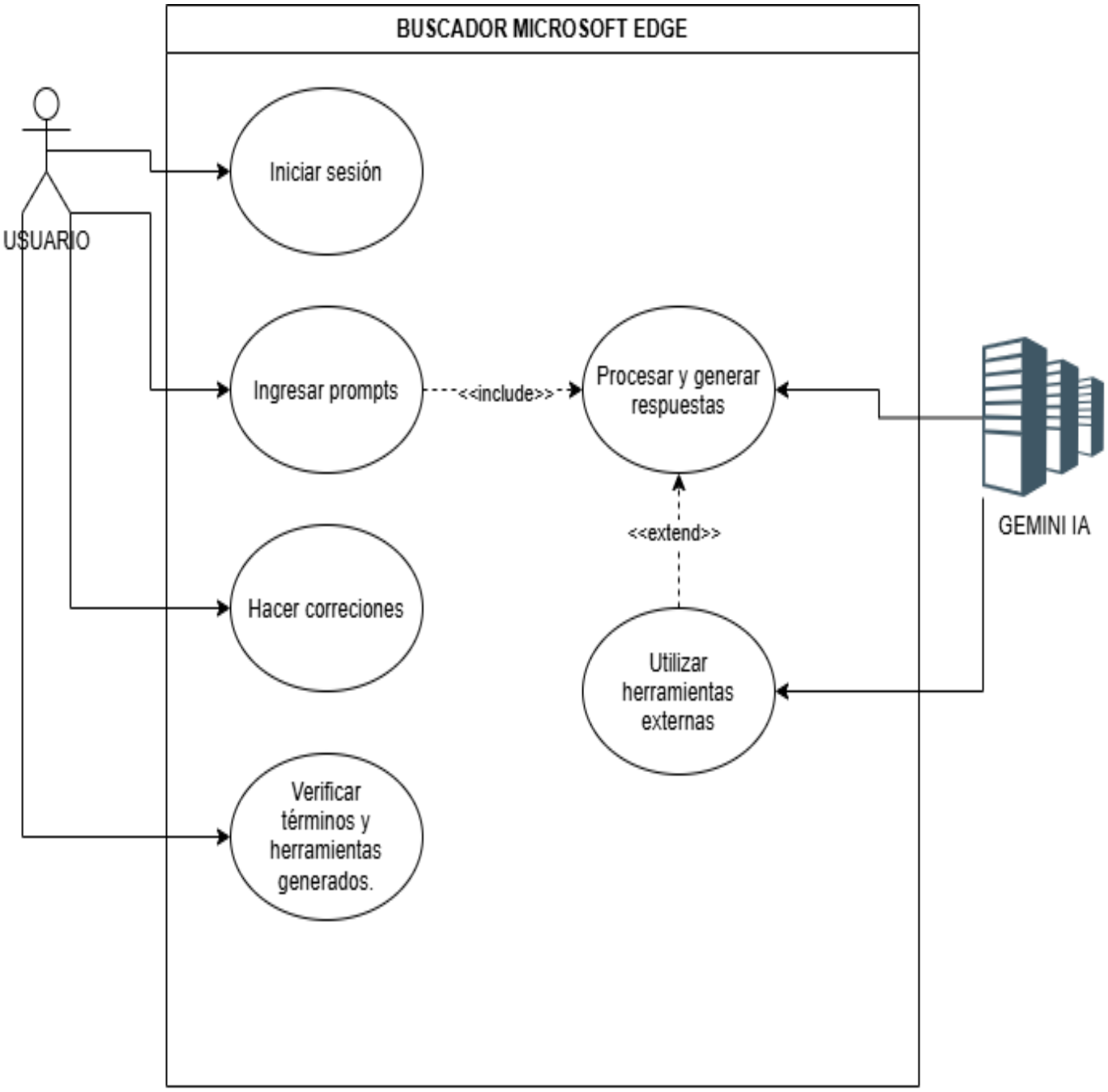
- Crear y probar el código que genere la herramienta, para posteriormente poder hacer la comparación con lo obtenido al hacer uso de la IA “GEMINI IA”.
- Crear y probar los prompts verificando los resultados de los mismos mediante pruebas y errores.
- Comparar los resultados obtenidos con ambos “métodos”.
- Generar las conclusiones de la investigación.
- Presentar un documento final.

FIGURA 1 (DIAGRAMA DE FLUJO)



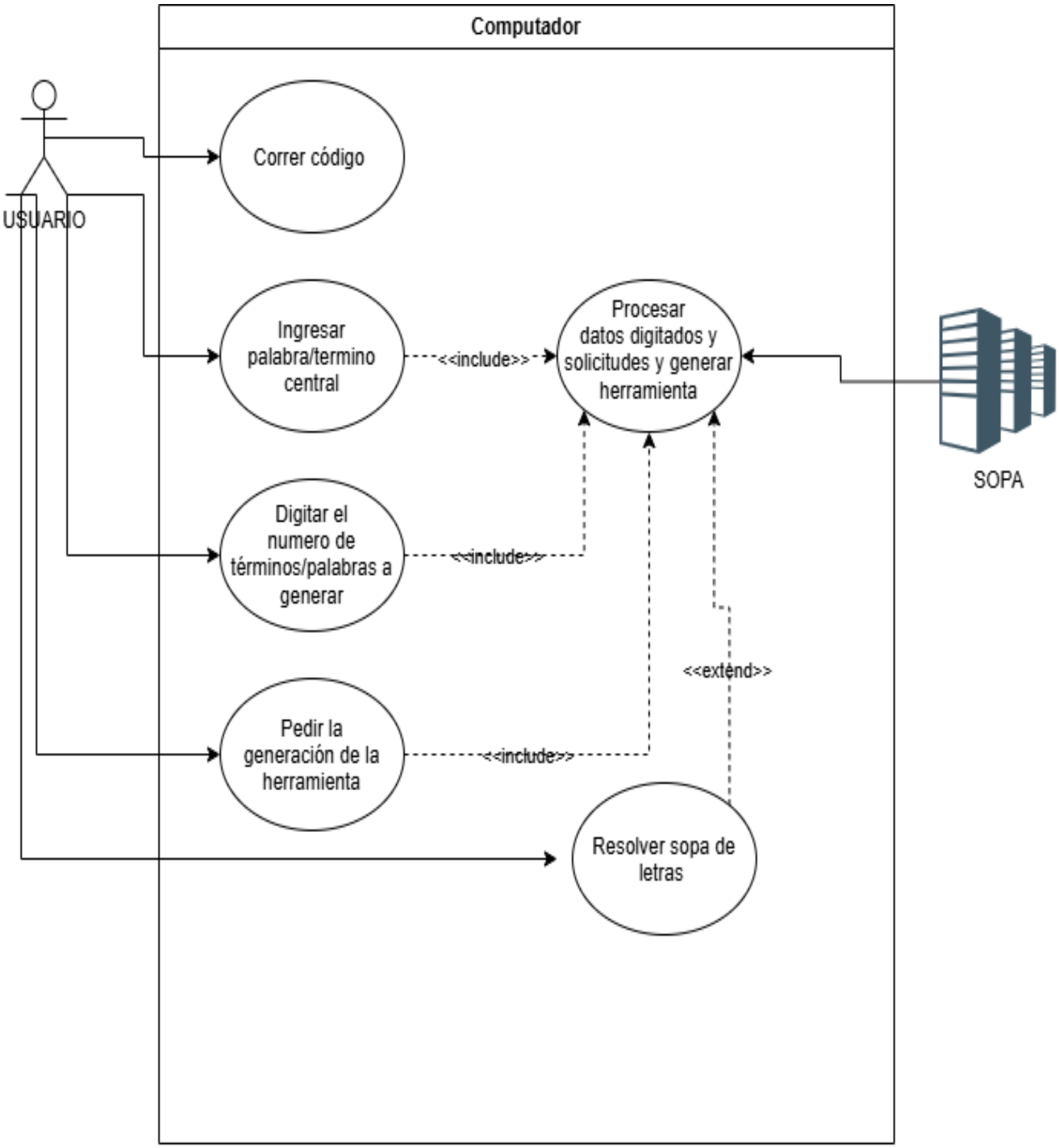
(*Nota.* Diagrama de flujo que ilustra las fases del proceso metodológico. Creación propia.)

FIGURA 2 (DIAGRAMA DE CASO DE USO EN EL BUSCADOR)



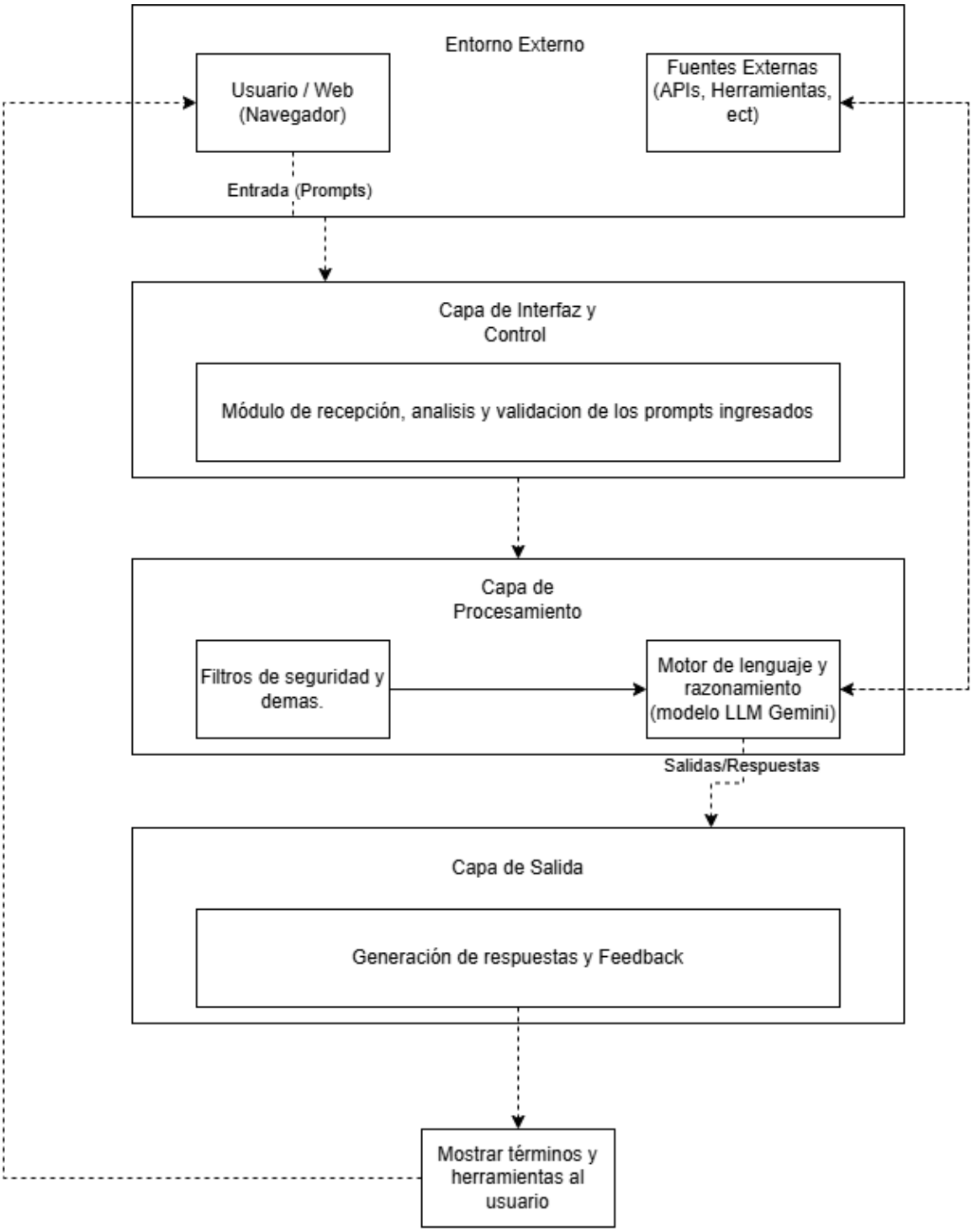
(*Nota.* Diagrama de casos de uso que representa la interacción entre el usuario y la plataforma Gemini IA en el proceso investigativo. Creación propia.

FIGURA 3 (DIAGRAMA DE CASO DE USO CODIGO PROPIO)



(*Nota.* Diagrama de casos de uso que representa la interacción entre el usuario y el código elaborado para hacer la comparación en el proceso investigativo. Creación propia.

FIGURA 4 (DIAGRAMA DEL SISTEMA)



(*Nota.* Diagrama del sistema hecho por capas que detalla el flujo de información desde la entrada hasta la presentación de respuestas. Creación propia.)

PLANTEAMIENTO Y USO DE PROMPTS

Recordando la idea del presente trabajo investigativo, es de mencionar que se hará uso de una versión paga de la IA “Gemini IA” para generar la sopa de letras a comparar con la que genere el programa. También recordar que la presente investigación se basa en determinar la capacidad que tiene la IA “Gemini” para en base a una palabra central, en este caso particular, (Redes), en el contexto y ámbito de la ingeniería de sistemas, generar una cantidad suficiente de términos que permitan a su vez que la misma IA genere una herramienta educativa como lo es la sopas de letras, dichas palabras bien pueden ser sinónimos, entendiendo el termino sinónimos como “una palabra o de una expresión: Que, respecto de otra, tiene el mismo significado o muy parecido, como *empezar* y *comenzar*.” (RAE) o merónimos, entendiendo que merónimo hace referencia a una “palabra cuyo significado mantiene, respecto del de otra, la misma relación que la parte respecto del todo”.(Rae) Por ejemplo: “tigre”, “lince”, “león” y “guepardo” son merónimos entre sí del holónimo “felino”, entendiendo que holónimo hace referencia a una “Palabra cuyo significado mantiene, respecto del de otra, la misma relación que el todo respecto de la parte.” (Rae). En resumen, para la presente investigación el holónimo es la palabra “Redes” en el contexto de la ingeniería de sistemas y las tecnologías de la información y por ende un posible sinónimo podría ser Redes de datos y un merónimo podría ser Router.

Ahora continuaremos con la primera idea de entrada/prompt a usar:

“Querido Gemini, necesito que generes una lista de 10 palabras que sean sinónimos o merónimos de la palabra “redes”, tomando únicamente como referencia el contexto de la Ingeniería de Sistemas y las tecnologías de la información.”
Con este primer prompt se obtuvo:

Sinónimos o conceptos equivalentes

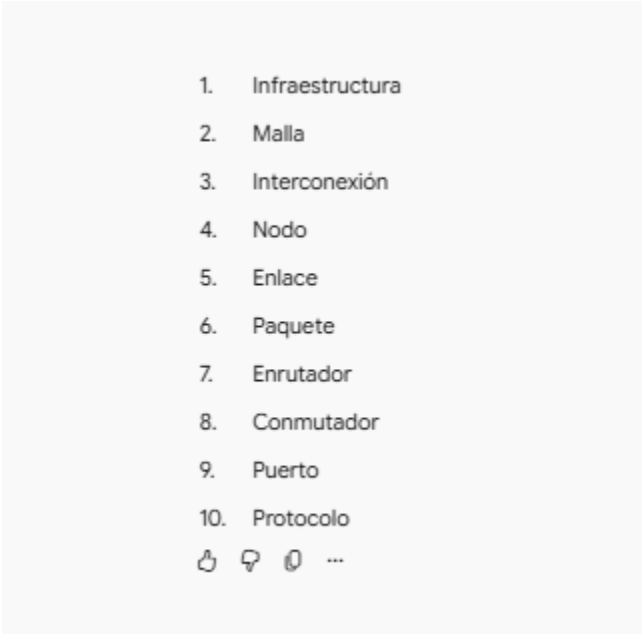
1. **Infraestructura** *(Sinónimo amplio)*: Conjunto de elementos físicos y lógicos que permiten el soporte, conectividad y transmisión de datos en un entorno informático.
2. **Arquitectura de comunicaciones** *(Sinónimo técnico)*: Estructura conceptual y organización lógica que define cómo interconectan y funcionan los sistemas de transmisión.
3. **Malla** *(Sinónimo/Variante de topología)*: Diseño o estructura de interconexión donde los nodos están conectados entre sí de forma directa o redundante.
4. **Interconexión** *(Sinónimo funcional)*: Sistema o conjunto de enlaces que unen múltiples dispositivos para permitir el intercambio de información.

Merónimos (Partes o componentes de una red)

1. **Enlace** *(Merónimo)*: Canal físico o inalámbrico (como un cable de fibra óptica o una radiofrecuencia) que conecta dos nodos en la red.
2. **Nodo** *(Merónimo)*: Punto de conexión o dispositivo activo (servidor, computadora, *switch*) capaz de enviar, recibir o reenviar paquetes de datos.
3. **Paquete** *(Merónimo)*: Unidad fundamental de datos formateada que viaja a través de la red de un origen a un destino.
4. **Enrutador** (*Router*) *(Merónimo)*: Dispositivo de hardware encargado de direccionar y guiar el tráfico de datos entre distintas redes.
5. **Puerto** *(Merónimo)*: Interfaz física o lógica a través de la cual se envían y reciben los datos dentro de un dispositivo de red.
6. **Protocolo** *(Merónimo del aspecto lógico)*: Conjunto de reglas y estándares lógicos que gobiernan la comunicación y el intercambio de datos entre los elementos de la red.

Aunque no es un mal inicio, recordemos que necesitamos una lista de los términos, mas no una explicación o definición de los mismos, haciendo la corrección y la aclaración el nuevo prompt queda así:

“Querido Gemini, necesito que generes una lista de 10 palabras que sean sinónimos o merónimos de la palabra “redes”, tomando únicamente como referencia el contexto de la Ingeniería de Sistemas y las tecnologías de la información. Dicha lista solo debe incluir las palabras que consideres oportunas al contexto, no debes definirlas ni nada similar.”
Con este prompt obtuvimos:



Como obtuvimos resultados positivos, procedemos con el prompt para que genere la herramienta:

“Ahora con estas palabras, haz una sopa de letras de tamaño 20 por 20 que sea como una imagen para imprimir.”
Con este prompt obtuvimos:

Nombre _____ Fecha _____

SOPA DE LETRAS: INGENIERÍA DE SISTEMAS Y TIC

P	R	I	N	F	R	A	E	S	T	R	U	C	T	U	R	A	J	P	D
P	R	I	N	F	R	A	E	S	T	R	U	C	T	U	R	A	J	P	D
A	J	L	E	Q	K	C	K	G	B	T	H	N	M	B	P	Q	L	F	K
Q	F	D	N	L	R	Y	O	M	K	I	R	M	B	T	L	W	E	Y	V
U	Z	N	T	M	G	O	N	F	S	R	C	B	X	P	K	C	N	A	B
E	F	G	E	R	M	A	R	X	I	L	T	G	L	R	G	M	J	I	Q
T	K	O	R	O	J	Q	E	X	L	G	D	H	Y	O	D	M	C	L	K
E	J	K	C	P	I	A	D	J	D	N	H	F	E	T	E	N	L	Y	F
Z	C	M	O	X	R	W	X	G	L	D	J	J	O	C	P	J	C	J	F
V	B	N	N	Y	Z	A	C	T	U	O	R	N	I	E	S	E	S	Y	F
A	R	E	E	D	N	B	Q	R	M	X	Z	M	L	L	D	G	T	H	H
B	N	S	X	I	K	O	G	R	B	H	W	J	O	N	F	W	Q	E	Y
F	T	N	I	L	R	C	Z	K	P	L	C	J	N	A	D	R	R	V	L
M	S	I	O	I	G	P	A	F	R	S	V	N	H	M	C	S	W	X	Y
L	P	T	N	B	D	D	F	Q	E	R	Y	N	B	C	D	Z	S	T	R
M	L	E	E	B	D	F	H	J	J	K	L	Y	I	Q	G	T	U	V	W
B	I	R	X	Z	A	N	B	N	T	R	G	D	L	B	F	B	N	M	Q
N	B	N	E	N	R	U	T	A	D	O	R	K	M	O	B	N	Q	X	Y
B	N	C	N	B	N	R	K	J	G	H	D	R	C	D	S	F	G	H	J
P	O	J	P	O	C	B	N	M	L	O	P	I	I	O	J	N	H	F	D
P	R	O	T	O	C	O	L	O	M	L	K	J	P	F	G	F	D	A	R

PALABRAS A BUSCAR

INFRAESTRUCTURA
MALLA
INTERCONEXIÓN
NODO
ENLACE
PAQUETE
ENRUTADOR
CONMUTADOR
PUERTO
PROTOCOLO

PARTES MAS RELEVANTES DEL CODIGO

Apartado de la Conexión con la API de Gemini

```
def obtener_palabras_gemini(self, palabra_inicial, cantidad):
    """Conecta con la API probando múltiples modelos en secuencia para evitar errores 503 por alta demanda."""
    api_key = self.entrada_api.get().strip()
    if not api_key:
        messagebox.showerror("Error", "Por favor ingresa tu API Key.")
        return []

    # Lista de modelos oficiales ordenados para alternar en caso de saturación
    modelos_a_probar = ["gemini-1.5-flash", "gemini-1.5-pro", "gemini-flash-latest"]

    prompt = (
        f"Genera exactamente {cantidad} palabras en español relacionadas estrictamente con el término '{palabra_inicial}'. "
        # ...
    )

    payload = {
        "contents": [
            {
                "parts": [
                    {
                        "text": prompt
                    }
                ]
            }
```

```
            ]
        }

        headers = {
            "Content-Type": "application/json",
            "X-goog-api-key": api_key
        }

        # Intenta con cada modelo disponible hasta que uno responda con éxito
        for modelo in modelos_a_probar:
            url =
f"https://generativelanguage.googleapis.com/v1beta/models/{modelo}:generateContent"
            try:
                response = requests.post(url, headers=headers, json=payload)

                if response.status_code == 200:
                    # ...
                    return resultado
            except Exception:
                continue
```

(Extrae la clave API, construye el payload del prompt indicando las instrucciones a Gemini y realiza la petición POST mediante la librería requests probando con distintos modelos.)

Apartado de la Ubicación de palabras y rellenado de espacios vacíos en la matriz

```
def colocar_palabra_en_sopa(self, palabra, tamano):
        """Intenta ubicar una palabra aleatoriamente en direcciones horizontal, vertical o diagonales."""
        direcciones = [(0, 1), (1, 0), (1, 1), (-1, 1)]
        colocada = intents = 0

        while not colocada and intents < 100:
            fila = random.randint(0, tamano - 1)
            col = random.randint(0, tamano - 1)
            dir_fila, dir_col = random.choice(direcciones)

            if self.puede_colocar_palabra(fila, col, dir_fila, dir_col, palabra, tamano):
                for i, letra in enumerate(palabra):
                    self.sopa[fila + i * dir_fila][col + i * dir_col] = letra
                colocada = True
            intents += 1

    def rellenar_espacios_vacios(self):
        """Rellena las casillas vacías de la matriz con letras aleatorias del abecedario."""
        letras = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
        for fila in range(len(self.sopa)):
            for col in range(len(self.sopa[fila])):
                if self.sopa[fila][col] == '':
                    self.sopa[fila][col] = random.choice(letras)
```

(colocar_palabra_en_sopa ubica las letras de las palabras recibidas en orientaciones aleatorias comprobando que no colisionen. Posteriormente, rellenar_espacios_vacios inserta caracteres al azar en los huecos sobrantes.)

Apartado de Renderizado de la matriz gráfica y comprobación de palabras del jugador

```
def mostrar_sopa(self, tamano):
        """Genera los botones visuales que forman la cuadrícula interactiva."""
        self.botones = []
        for fila in range(tamano):
            fila_botones = []
            for col in range(tamano):
                boton = tk.Button(self.grid_frame, text=self.sopa[fila][col], width=3, height=1,
                                  font=("Arial", 10, "bold"),
                                  command=lambda f=fila, c=col: self.seleccionar_casilla(f, c))
                boton.grid(row=fila, column=col, padx=1, pady=1)
                fila_botones.append(boton)
            self.botones.append(fila_botones)

    def seleccionar_casilla(self, fila, col):
        """Maneja la selección interactiva de casillas por parte del usuario."""
        boton = self.botones[fila][col]
        if (fila, col) in self.seleccion_actual:
            boton.config(bg="SystemButtonFace")
            self.seleccion_actual.remove((fila, col))
        else:
            boton.config(bg="yellow")
            self.seleccion_actual.append((fila, col))

        self.verificar_palabra()

    def verificar_palabra(self):
```

```
        """Verifica si la selección actual del usuario corresponde a una palabra de la lista."""
        palabra_formada = ''.join([self.sopa[f][c] for f, c in self.seleccion_actual])

        if palabra_formada in self.palabras or palabra_formada[::-1] in self.palabras:
            palabra_encontrada = palabra_formada if palabra_formada in self.palabras else palabra_formada[::-1]

            for f, c in self.seleccion_actual:
                self.botones[f][c].config(bg="green")

            self.palabras_encontradas.add(palabra_encontrada)
            self.seleccion_actual = []
            self.actualizar_lista_palabras_visual()
```

(Crea la cuadrícula de botones interactivos con Tkinter, actualiza el color de selección según los clics e identifica si la palabra seleccionada (normal o invertida) pertenece a la lista para pintarla de verde.)

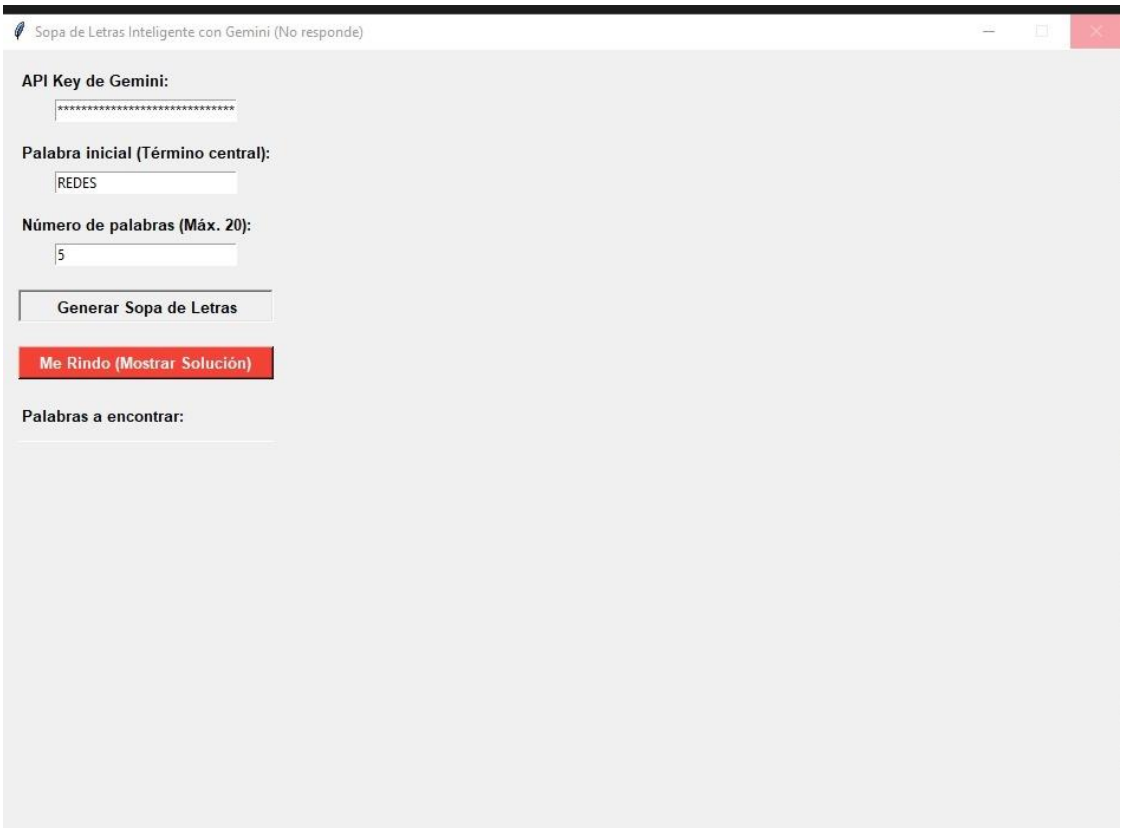
Aparatado de “me rindo”

```
def mostrar_palabras(self):
    """Revela la solución completa marcando las palabras en rojo."""
    for palabra in self.palabras:
        for fila in range(len(self.sopa)):
            for col in range(len(self.sopa[fila])):
                for dir_fila, dir_col in [(0,1), (1,0), (1,1), (-1,1)]:
                    if self.puede_colocar_palabra_solucion(fila, col, dir_fila, dir_col, palabra):
                        for i in range(len(palabra)):
                            f = fila + i * dir_fila
                            c = col + i * dir_col
```

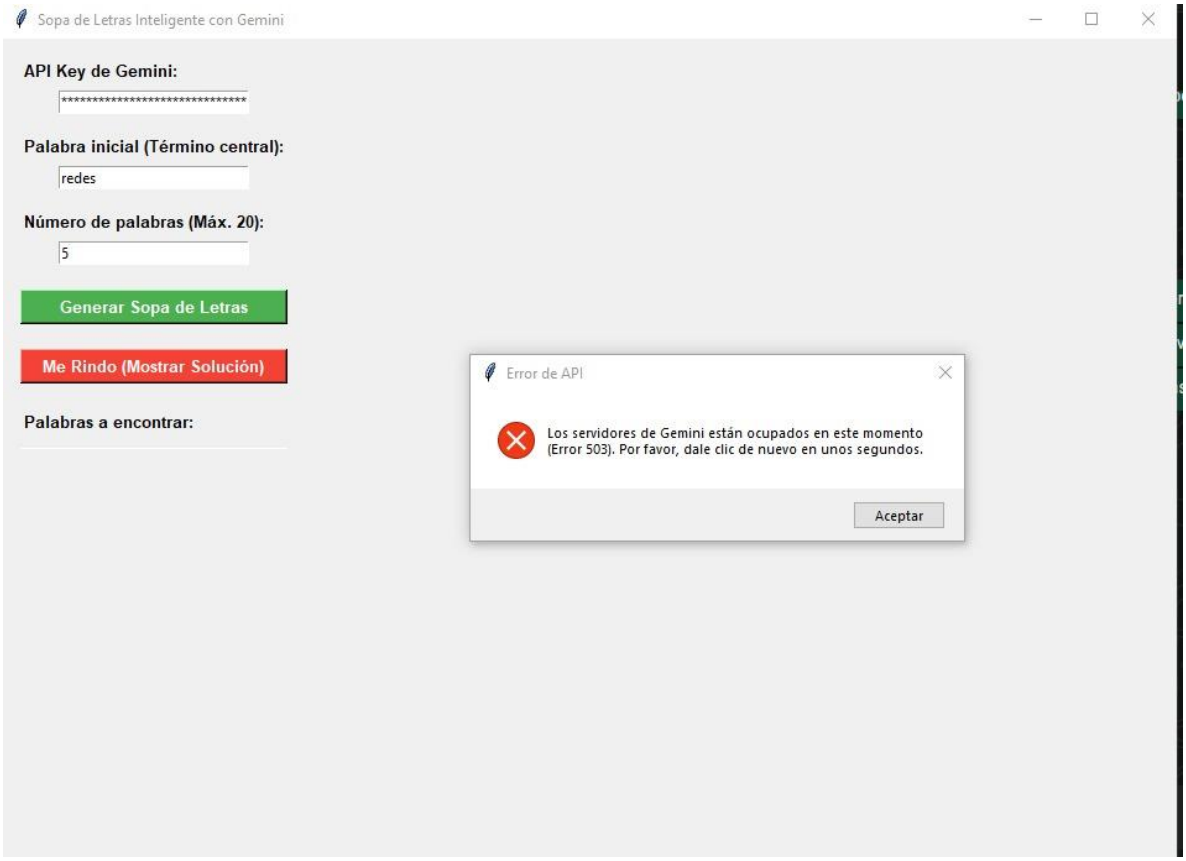
```
                    self.botones[f][c].config(bg="red")
```

(Recorre las palabras objetivo buscando su coincidencia en el tablero e ilustra visualmente sus casillas en color rojo.)

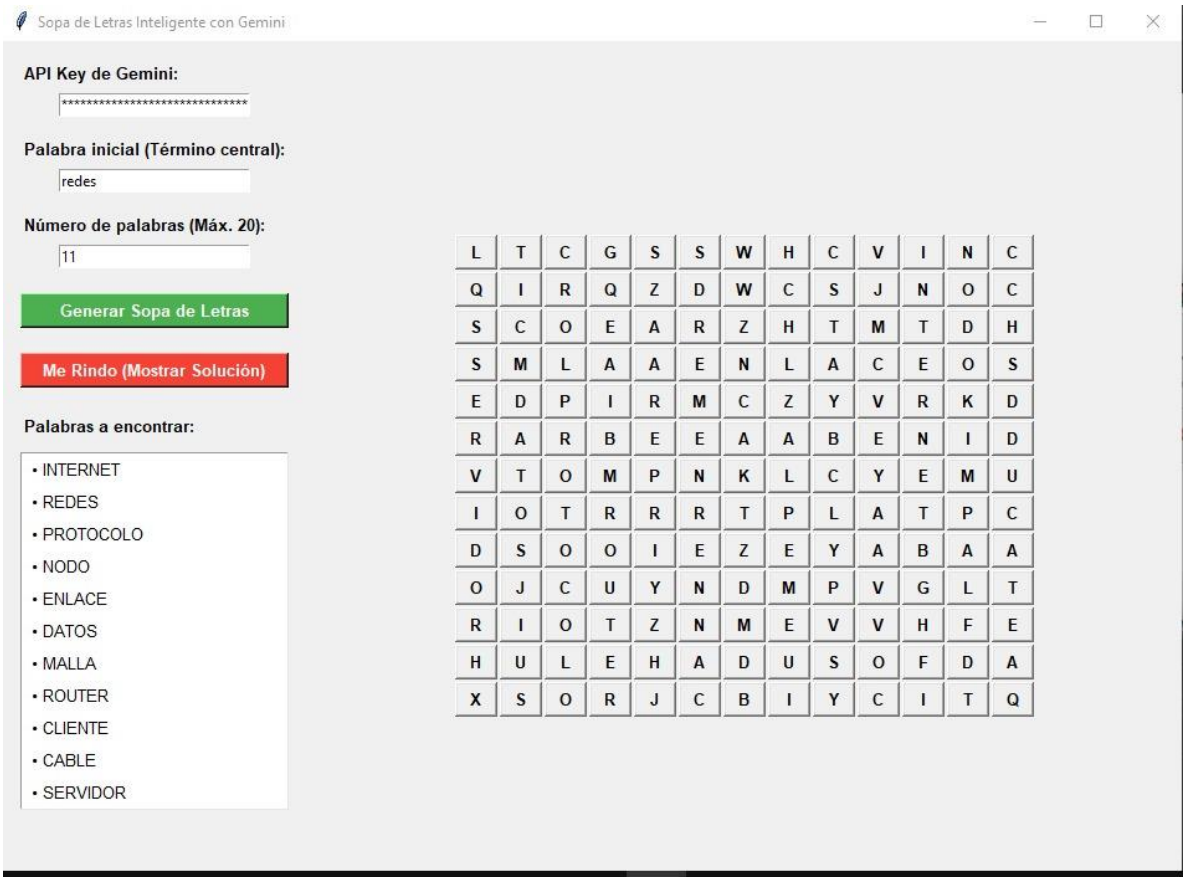
EVIDENCIAS DE LO OBTENIDO CON EL CODIGO



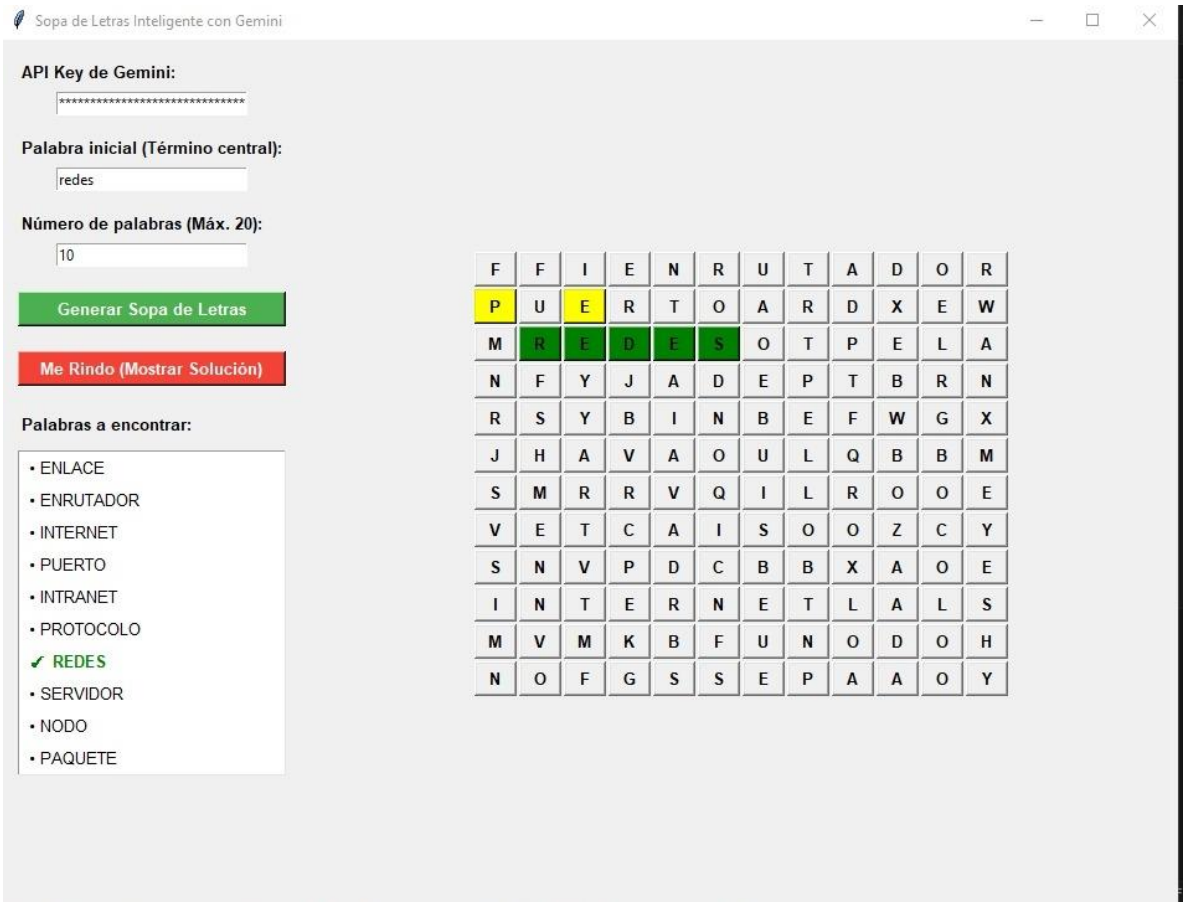
(Pantallazo posterior a correr el código, en la imagen se observa la interfaz a la que accede el usuario)



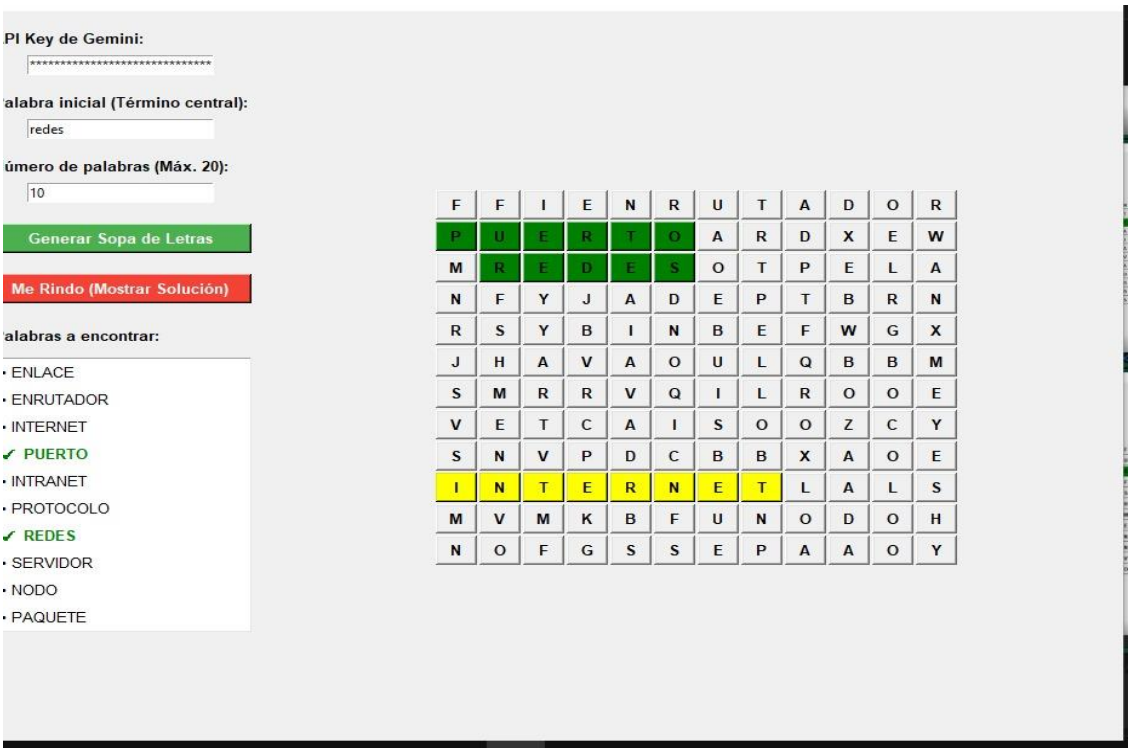
(Pantallazo del único pero enorme inconveniente encontrado durante el proceso, en la mayoría de los casos la herramienta no se genera por ese “error de api”, que hace referencia a que los servidores a los que intenta acceder el programa para generar las palabras y posteriormente poder generar la sopa de letras están ocupados)



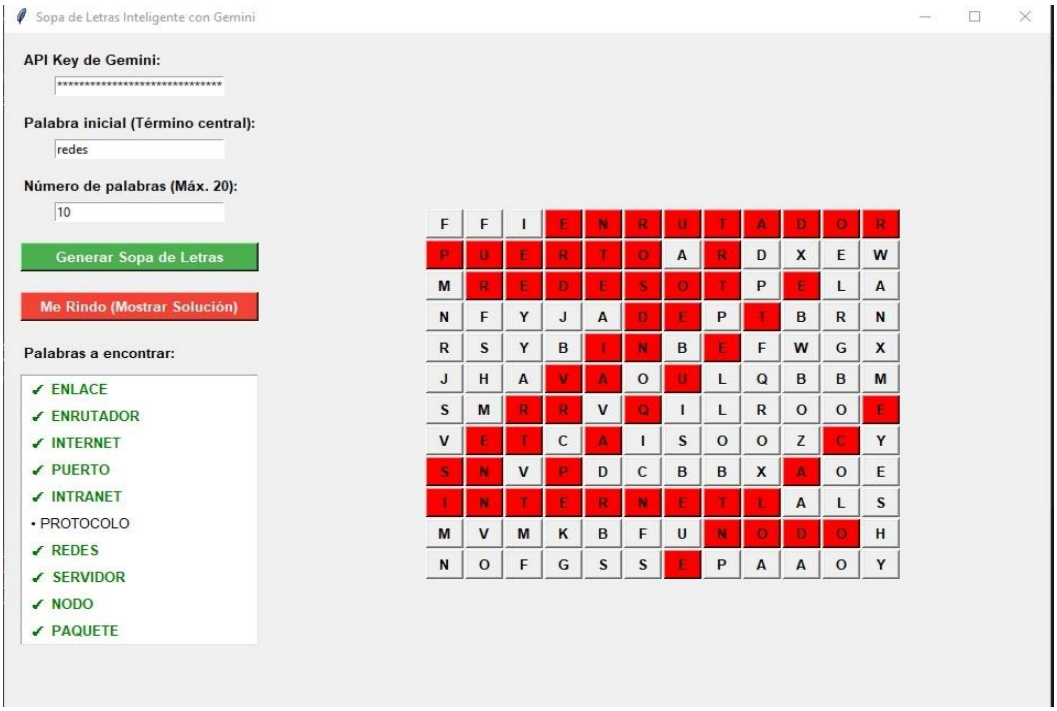
(Pantallazo que evidencia el funcionamiento del programa, aquí se ven los términos y la sopa de letras generada)



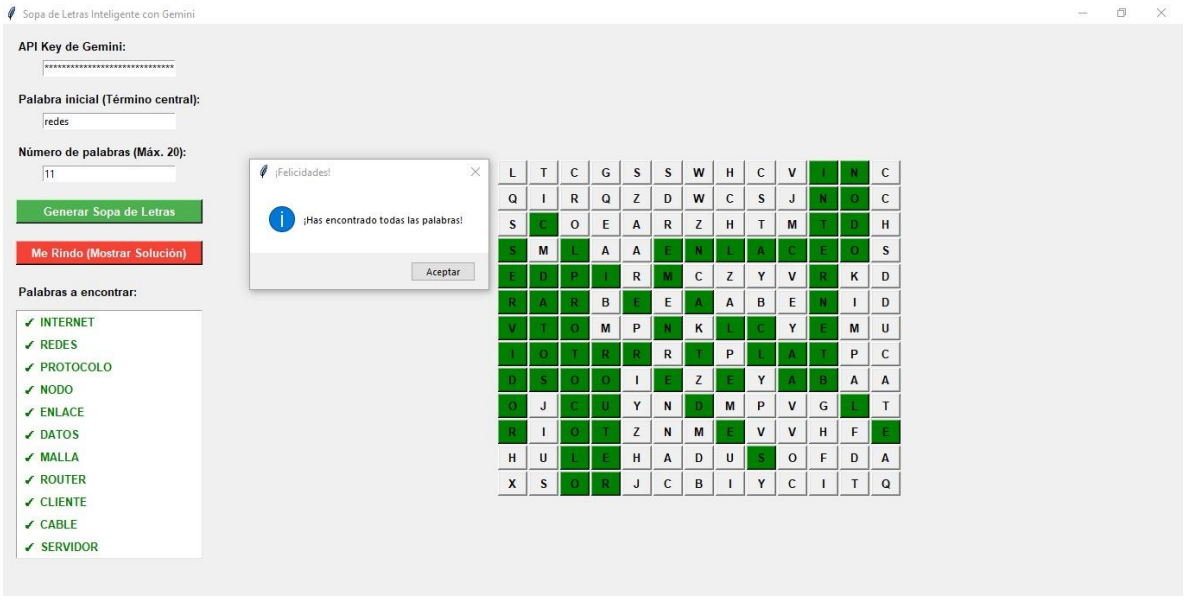
(Pantallazo de la interacción con el programa, como se puede ver, al momento de encontrar una palabra e iniciar a marcarla, las letras se tiñen de amarillo, al terminar de marcarla esta se tiñe de verde tanto en la sopa como en el recuadro de las palabras y aparece un chulito junto a la misma)



(Pantallazo en donde se puede ver que, si se encuentra una palabra, pero no se marcan las letras en orden, esta no se da por valida, sigue en amarillo)



(Pantallazo que permite ver lo que sucede cuando se hace Click en la opción de “Me Rindo (Mostrar Solución)”)



(Pantallazo que permite ver el mensaje que se muestra al resolver la sopa de letras)

TABLA COMPARATIVA

La siguiente tabla muestra las ventajas y desventajas del proceso y resultado obtenido con la IA “GEMINI IA” utilizada desde el buscador de Microsoft Edge frente al código hecho:

VENTAJAS	DESVENTAJAS
Velocidad de Generación: Debido al error del api, se deben hacer demasiados intentos para que el programa corra y genere la herramienta, mientras que la IA desde la interfaz del buscador la genera tan pronto se le digita el prompt.	**Interacción:** El programa permite interactuar con la sopa de letras tan pronto la genera, mientras que a través del buscador se tendría que imprimir la imagen o algo similar para interactuar con la sopa de letras.
Estética: La imagen generada por la IA “GEMINI IA” desde la interfaz del buscador es más agradable (más bonita) que la interfaz del programa.	**Acceso:** La sopa de letras generada por la IA a través de la interfaz del buscador presentada en este documento se hizo haciendo uso de la versión paga de la misma, quizás al momento de usar la versión gratuita de la IA, esta se tarde más o directamente no la genere.
Formato: La IA desde el buscador genera plantillas diagramadas con encabezados ("Nombre", "Fecha"), bordes e ilustraciones temáticas listas para imprimir, mientras que la interfaz del programa se limita a una interfaz.	**Personalización de parámetros:** El programa permite definir la palabra central, la cantidad exacta de palabras y cuenta con un listado interactivo mientras que la imagen de la IA requiere solicitar el contenido mediante prompt en texto plano.

Facilidad de Uso: El uso de la IA desde el buscador no requiere instalaciones de software, manejo de código ni configuración de llaves de acceso, mientras que el programa requiere muchos pasos previos (instalación de programas y demás),	**Falta de Control en la Generación de Palabras:** Se necesita hacer prompts muy específicos para que La IA desde el buscador no incluya palabras no deseadas, términos repetidos o con errores tipográficos dentro de la sopa de letras al generar la imagen, mientras que el programa garantiza mediante código un control exacto y sin errores en la inserción de la lista de palabras.

CONCLUSIONES

- Se demostró que Gemini es una herramienta bastante eficaz para entender un concepto central (en este caso, *Redes*) y generar automáticamente una lista de palabras directamente relacionadas con su contexto (como *Nodo*, *Router*, *Protocolo* o *Enlace*). La IA demostró buena coherencia semántica al diferenciar entre términos generales y partes específicas del sistema, lo que valida su utilidad para crear contenido educativo de forma rápida.

- Se logró crear con éxito una aplicación propia en Python que se conecta con la IA para obtener las palabras y construir la sopa de letras en una pantalla interactiva. A pesar de que este programa no funciona siempre debido al error de api, se cumplió con la meta de ofrecer un recurso funcional donde el usuario puede dar clic, marcar palabras y ver su progreso en tiempo real.

- A través del método de prueba y error, se evidenció que la forma en que se le pide la información a la IA es crucial. En los primeros intentos, la IA entregaba explicaciones largas que no servían para la sopa de letras, pero al ajustar el *prompt* de forma clara y directa, se logró que devolviera únicamente la lista limpia de palabras necesaria para el programa.

- Al comparar la IA en el buscador frente al programa propio, se identificaron ventajas y desventajas claras en ambos lados:

  El buscador: Gana en estética y rapidez visual, entregando una imagen decorada y lista para imprimir de inmediato.

  El programa propio: Gana en interactividad y control, permitiendo jugar directamente en el computador y configurar parámetros exactos (como la cantidad de palabras), aunque depende de la estabilidad de la API para no generar errores.

- Con la elaboración de la tabla comparativa, el análisis de los diagramas de flujo y casos de uso, y la consolidación de las pruebas realizadas, se completa la entrega del documento de investigación, cumpliendo así con la totalidad de las metas planteadas al inicio del trabajo.

BIBLIOGRAFIA

- Definiciones del Diccionario (RAE):

  Real Academia Española. (s.f.). Holónimo. En *Diccionario de la lengua española*. Recuperado el 20 de octubre de 2026, de https://dle.rae.es/hol%C3%B3nimo

  Real Academia Española. (s.f.). Merónimo. En *Diccionario de la lengua española*. Recuperado el 20 de octubre de 2026, de https://dle.rae.es/mer%C3%B3nimo

  Real Academia Española. (s.f.). Sinónimo. En *Diccionario de la lengua española*. Recuperado el 20 de octubre de 2026, de https://dle.rae.es/sin%C3%B3nimo